TIG-101 : recalcul local d'une tournée

Documentation du ticket « Cas 4 – Recalcul local d'une tournée pour un ordre imposé (A*, sans relancer le TSP) » – projet Optimod'Lyon

1. Pourquoi ce ticket

Décision du 26/10 : **le voyageur de commerce (TSP) ne sert qu'au bouton « Calculer »**. Il peut prendre jusqu'à 10 s. Toute modification faite ensuite (supprimer, ajouter, déplacer, réordonner, changer de livreur, restaurer) doit garder l'ordre des autres arrêts et ne recalculer que les trajets et les heures. Ce ticket fournit la **brique commune** qui fait ce recalcul, et l'utilise tout de suite pour la suppression d'une demande.

Avant : après une suppression, la tournée affichée disparaissait et il fallait recliquer sur « Calculer » (donc relancer le TSP). Maintenant : si une tournée est affichée, elle est mise à jour immédiatement.

2. Ce qui a été ajouté ou modifié

Fichier	Changement	Pourquoi là
model/Plan.java	2 méthodes publiques : `recalculerTournee` et `calculerTourneeSansDemande` ; 2 méthodes privées `trouverRangArret` et `dureeSurPlace`	Règle du nettoyage : A* et les heures de passage sont des méthodes privées de Plan, pas de nouvelle classe.
controller/MainController.java	`memoriserTourneeSansDemande(position)`	Le contrôleur garde le cache des tournées (une tournée par liste de demandes).
controller/EtatTourneeCalculee.java	redéfinit `supprimerDemande`	Design pattern State : c'est l'état « tournée affichée » qui sait quoi faire.
PlanTest, MainControllerTest	10 + 4 nouveaux tests	Tests écrits d'abord (exigence du cours).
README.md, docs/diagrams/classes.puml	utilisation, architecture, diagramme de classes	AGENTS.md : doc dans la même PR que le code.

3. Plan.recalculerTournee : la brique commune

`recalculerTournee(LocalTime heureDepart, List<Noeud> adresses, List<Integer> durees)`

- **Entrée** : les adresses dans l'ordre de passage, **de l'entrepôt à l'entrepôt**, et la durée sur place de chacune (0 pour l'entrepôt).
- **Sortie** : une `Tournee` (arrêts avec heures d'arrivée et de départ, chemins entre arrêts consécutifs).
- **Règles** identiques à `calculerTournee` : 15 km/h, retour avant 13 h sinon `IllegalStateException` au message lisible ; adresse inaccessible : même exception.
- **Erreur de programmation** (listes vides, tailles différentes, tournée qui ne revient pas à l'entrepôt) : `IllegalArgumentException`.

Comment c'est fait (le code est commenté pas à pas) :

- On retire le retour à l'entrepôt de la liste : l'étape des heures de passage le rajoute toute seule, comme dans le calcul complet.

- On calcule par A* le chemin **seulement entre arrêts consécutifs** (n trajets) au lieu de tous les couples (n^2) comme le calcul complet. C'est tout ce dont les heures de passage ont besoin.
- On réutilise `plusCourtChemin` (A*) et `calculerHeuresDePassage` **sans les modifier**. Le tableau de chemins passé à cette seconde méthode est rempli seulement sur les cases utilisées ; les autres restent à `null` (voir « Points d'attention »).

Exemple vérifié à la main (petit réseau de test), ordre imposé 1 → 2 → 3 → 1, départ 08:00, durées 60 s et 120 s :

Étape	Distance	Temps à 15 km/h	Arrivée	Départ
1 → 2 (détour par 3)	249,6 m	59,9 s, arrondi 60 s	08:01:00	08:02:00
2 → 3 (par 1)	400 m	96 s	08:03:36	08:05:36
3 → 1 (par 2)	449,6 m	107,9 s, arrondi 108 s	08:07:24	08:07:24

L'ordre est respecté même s'il est plus long : l'ordre optimal 1 → 3 → 2 → 1 revient à 08:05:12.

4. Plan.calculerTourneeSansDemande : retirer une demande

`calculerTourneeSansDemande(Tournee tournee, Demande demande)` retire de la tournée les deux arrêts de la demande (collecte et livraison), garde l'ordre de tous les autres, puis appelle `recalculerTournee`.

- Une tournée ne retient pas la demande de chaque arrêt. Les deux arrêts sont donc retrouvés par leur **nœud** et leur **durée sur place** (heure de départ moins heure d'arrivée). Le départ et le retour (entrepôt) ne sont pas cherchés.
- **Cas ambigu** : si deux arrêts ont le même nœud et la même durée (deux demandes avec le même point de collecte), on ne sait pas lequel retirer sans risquer de casser la contrainte « collecte avant livraison » d'une autre demande. La méthode lève alors `IllegalArgumentException`. Les fichiers fournis n'ont jamais ce cas (vérifié), mais l'utilisateur peut le créer en cliquant deux fois le même nœud.
- Retirer des arrêts ne peut pas allonger une tournée possible (inégalité triangulaire des plus courts chemins), donc l'échec 13 h n'est pas attendu ici.

5. Le branchement dans l'application

```
Clic « Supprimer la demande »  (une tournée est affichée)
  EtatTourneeCalculee.supprimerDemande(controleur, position)
    1. controleur.memoriserTourneeSansDemande(position)
         - tournée actuelle = cache[demandes actuelles]
         - nouvelle = plan.calculerTourneeSansDemande(actuelle, demande)
         - cache[demandes restantes] = nouvelle   (sauf cas ci-dessous)
    2. super.supprimerDemande(...)     // comportement habituel
         - CommandeSupprimerDemande exécutée (annulable, Observer notifié)
         - retrouverEtatProgramme() trouve la tournée dans le cache
           -> EtatTourneeCalculee + afficherTournee(nouvelle)
```

On réutilise le cache existant plutôt que d'ajouter un nouveau mécanisme : **annuler (Ctrl+Z) et rétablir (Ctrl+Y)** fonctionnent donc sans rien de plus (la tournée complète et la tournée réduite sont toutes deux en cache).

On ne range rien dans le cache, et la tournée est simplement retirée de l'écran comme avant, dans ces cas :

- position invalide ;
- la tournée des demandes restantes a déjà été calculée par le TSP (elle est meilleure, on la garde) ;

- le recalcul local est impossible (arrêts ambigus).

6. À quoi c'est relié

Ticket	Ce qu'il fait de cette brique
TIG-95 – plusieurs livreurs	Il définira le modèle multi-livreurs. `recalculerTournee` a été écrit sur le modèle actuel (une tournée) ; il faudra l'appeler une fois par livreur et suivre le contrat publié par LK et Le J (Arret enrichi, tournées dans le modèle).
TIG-102 – ajout par insertion	Choisit la position d'insertion, puis appelle `recalculerTournee` avec le nouvel ordre.
TIG-43 / TIG-44 – déplacer, réordonner	Construisent le nouvel ordre, puis appellent `recalculerTournee`.
TIG-80 – changer de livreur	Retire l'arrêt d'une tournée et l'insère dans une autre : deux recalculs.
TIG-56 – restauration	Recalcule les tournées sauvegardées sans relancer le TSP.

7. Tests (écrits avant le code)

Test	Ce qu'il prouve
ordreDuTspRedonneLesMemesHeures	Fichiers fournis (petit et moyen plan) : l'ordre du TSP redonne exactement la tournée de calculerTournee.
ordreImposeRespecteMemeSiPlusLong	Heures calculées à la main (tableau du § 3).
recalculRetourApres13hRefuse, recalculAdresseInaccessibleRefusee, recalculRefuseDesListesIncoherentes	Les trois erreurs.
recalculSansArretReduitAuDepart	Une tournée sans arrêt = le départ seul.
suppressionGardeLOrdreDesAutresArrets	Pour chaque demande du fichier moyen : même ordre qu'avant moins ses 2 arrêts, mêmes heures qu'un recalcul, retour jamais plus tard.
suppressionDeLaDerniereDemande…, suppressionDUneDemandeAbsente…, suppressionRefuseUnArretAmbigu	Cas limites.
MainControllerTest (4 tests)	Pas de nouveau calcul complet (compteur), tournée affichée, annuler/rétablir via le cache, position invalide, tournée du TSP non écrasée.

8. Points d'attention

- **Cases « null » du tableau de chemins** dans recalculerTournee : choix pour réutiliser calculerHeuresDePassage sans la modifier. Ces cases ne sont jamais lues ; c'est expliqué en commentaire. Une refonte plus propre serait de faire prendre à calculerHeuresDePassage directement la liste des chemins consécutifs.
- **Optimisation possible** : on recalcule par A* tous les trajets consécutifs, alors que seuls les trajets touchés changent. Le ticket demande la version simple ; le coût reste de l'ordre de 20 A* sur le grand plan.
- **Adaptation à TIG-95** à prévoir dès que le contrat est publié.

- **Vérifications** : tous les tests (115) passent ; les lignes font au plus 80 caractères ; la Javadoc ne produit aucune erreur. Maven Central étant inaccessible depuis l'environnement de travail, `mvn clean verify`, Checkstyle, SpotBugs et JaCoCo n'ont **pas** pu être lancés : à faire en local avant la PR.

9. Vérifier à la main

- Lancer l'application, importer `grandPlan.xml` et `demandeGrand9.xml`, cliquer sur « Calculer ».
- Sélectionner une demande, « Supprimer la demande » : la liste des arrêts à droite se met à jour tout de suite, sans l'attente du calcul.
- Ctrl+Z puis Ctrl+Y : les deux tournées réapparaissent sans nouveau calcul.
- Ajouter une demande : la tournée est retirée (l'ajout reste à faire dans TIG-102).